Minimizing fStress by Majorizing A Gauss-Newton Approximation

Jan de Leeuw

April 3, 2026

TBD

Table of contents

Note: This is a working manuscript which will be expanded/updated frequently. All suggestions for improvement are welcome. All Rmd, tex, html, pdf, R, and C files are in the public domain. Attribution will be appreciated, but is not required. The files can be found at https://github.com/deleeuw/rStress

1 Loss Functions

Metric Multidimensional Scaling (MDS) minimizes the loss function *(raw) stress*, originally proposed by Kruskal (1964a), Kruskal (1964b). Stress is

$$\sigma(x) := \sum_{k=1}^{K} w_k(\delta_k - d_k(x))^2, . \tag{1}$$

In definition (1)

- w is a vector of K known positive *weights*;
- δ is a vector of K known non-negative *dissimilarities*;
- $x := \text{vec}(X)$, with X the *configuration*, an $n \times p$ matrix of n *points* in p *dimensions*;
- $d(x)$ are K Euclidean *distances* between some or all pairs of points.

Thus $K \leq \frac{1}{2}n(n-1)$. Our notation is slighty non-standard, because we use vectors in our definition, and not matrices. For each $1 \leq k \leq K$ there is a pair of indices $(i(k), j(k))$ with $1 \leq i(k) < j(k) \leq n$. The unit vectors $e_{i(k)}$ and $e_{j(k)}$ are used to define the matrix A_k, the direct sum of p copies of $(e_{i(k)} - e_{j(k)})(e_{i(k)} - e_{j(k)})'$. Suppose $n = 4$ and $p = 2$ and $(i(k), j(k))$ is $(2, 4)$. Then

$$A_k = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & +1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & +1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & +1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & +1 \end{bmatrix}. \tag{2}$$

The A_k, together with $x = \text{vec}(X)$, allow us to conveniently write $d_k(x) = \sqrt{x' A_k x}$.

In non-metric MDS stress is a function of both x and δ, and we minimize over both arguments. We require that $\delta \in \Delta$, with Δ is subset of the non-negative orthant of $\mathbb{R}^K$. In ordinal MDS, the most common case, Δ is defined as the cone of vectors with $0 \leq \delta_1 \leq \cdots \leq \delta_k$, together with some normalization requirement to exclude the trivial solution with both x and δ (and thus stress) equal to zero. The simplest such requirement is that δ is on the sphere $\mathbb{S}$ defined as the vectors δ for which

$$\sum_{k=1}^{K} w_k \delta_k^2 = 1. \tag{3}$$

1.1 Metric fStress

Groenen, De Leeuw, and Mathar (1995) propose a generalization of stress called *fStress*. It is defined as

$$\sigma_f(x) := \sum_{k=1}^K w_k(f(\delta_k) - f(d_k(x)))^2, \tag{4}$$

with f a known real-valued function defined on $(0, +\infty)$. We assume that

1. f is strictly increasing on $(0, +\infty)$,
2. f is continuously differentiable on $(0, +\infty)$.

Thus we do *not* require f to be defined at zero. We also do *not* require, but we have a preference for functions which have, in addition,

3. $f(0) = 0$,
4. $\lim_{x\to\infty} f(x) = \infty$,
5. f is positively homogeneous, i.e. $f(\lambda x) = \lambda^\mu f(x)$ for all $\lambda \geq 0$ and for some $\mu > 0$.
6. f is convex.

Note that (5) implies (3) and (4). Power functions x^r with $r > 0$ clearly satisfy the first five required and preferred conditions, and the sixth one if $r \geq 1$. We will also consider the example of the logarithm (Ramsay (1977)), which does not satisfy preferred conditions (3),(5) and (6). An example of an f that does not satisfy (4), (5) and (6) is the arctangent, bounded by $\pi/2$ as $x \to \infty$. The logarithm, the arctangent, and the power function with $r \leq 1$ are all concave.

For our purposes it is important that Groenen, De Leeuw, and Mathar (1995) show that at a local minimum of fStress we have $d_k(x) > 0$ for all k for which $w_k\delta_k > 0$, generalizing the result for stress proved by De Leeuw (1984). Thus, under weak conditions on the weights and dissimilarities, fStress is differentiable at local minima. Groenen, De Leeuw, and Mathar (1995) give no explicit algorithm to minimize fStress, but they do derive formulas for the first and second derivatives, which can of course be used in general-purpose optimization routines.

The important special case of fStress in which f is a power function is called *rStress* (also known as *powerStress*). It is

$$\sigma_r(x) := \sum_{k=1}^K w_k(\delta_k^r - d_k^r(x))^2 \tag{5}$$

If $r = 1$ we have Kruskal's stress, if $r = 2$ we have *sstress* used by Takane, Young, and De Leeuw (1977). Using

$$\log(x) = \lim_{r\to 0} \frac{x^r - 1}{r} \tag{6}$$

shows that the logarithmic loss function used by Ramsay (1977) is a limiting case of rStress.

There have been various attempts to extend the smacof majorization (or MM) method for MDS (De Leeuw (1977)) to rStress. References and links to various unpublished reports are in De Leeuw (2017). The recent smacofx package (Rusch et al. (In Press)) has R code for the rstressMin() function that implements the majorization method in De Leeuw, Groenen, and Mair (2016).

Minimizing (4) can be interpreted as approximating dissimilarities δ by distances $d(x)$ with a weighted least squares loss function, with f incorporated in the weights. This can be seen most clearly if the fit is good, i.e. when δ and $d(x)$ are close. We can then approximate fStress from (4) as in

$$\sigma_f(x) \approx \sum_{k=1}^K w_k\{f'(\delta_k)\}^2(\delta_k - d_k(x))^2. \tag{7}$$

By modifying the weights in this way (Groenen and Van de Velden (2016)) minimizing (7) can be done by standard metric smacof. This will presumably give a good approximation to minimizing fStress if the fit is good.

1.2 Non-metric fStress

In non-metric MDS we minimize fStress not only over x but also over δ, where δ is constrained to be in some subset Δ of $\mathbb{R}^K$.

The most common case is ordinal MDS, where Δ is defined by the inequalities $\delta_1 \leq \cdots \leq \delta_k$. In the ordinal case we redefine fStress as

$$\sigma_f(x, \hat{d}) := \sum_{k=1}^K w_k(\hat{d}_k - f(d_k(x)))^2, \tag{8}$$

and we require that $\hat{d} \in \Delta \cap S$, where S is the sphere

$$\sum_{k=1}^K w_k \hat{d}_k^2 = 1. \tag{9}$$

In the MDS literature the $\hat{d}_k$ are called the *disparities* or the *pseudo-distances*. Note that it does not make a difference if we require the disparities $\hat{d}$ to be monotone with the dissimilarities δ or with the transformed dissimilarities $f(\delta)$.

It is important to observe that for rStress we have homogeneity

2 Algorithm

The technique proposed in this paper to minimize non-metric fStress is in the Alternating Least Squares (ALS) family. We start with an initial estimate $x^{(0)}$. We then minimize σ_f over δ in $\Delta \cap \Sigma$ for the current $d(x^{(0)})$. The minimizer $\delta^{(0)}$ is then used to minimize fStress over x for the current disparities, yielding $x^{(1)}$. These two steps are alternated until x and δ do not change any more. Starting in iteration $\nu = 0$ we compute

$$\delta^{(\nu)} = \underset{\delta \in \Delta \cap \Sigma}{\operatorname{argmin}} \, \sigma_f(x^{(\nu)}, \delta), \tag{10a}$$

$$x^{(\nu+1)} = \underset{x}{\operatorname{argmin}} \, \sigma_f(x, \delta^{(\nu)}). \tag{10b}$$

We then increase ν by one and go into the next iteration. And so on, until convergence.

2.1 Normalized Cone Regression

In metric MDS there is no need for the first step (10a), because $\delta^{(\nu)}$ is equal to the normalized dissimilarities δ for all ν. In the non-metric case step (10a) is needed, but compared to (10b) it is comparatively easy. We have to compute the least squares projection of $f(d(x))$ on the cone Δ and then normalize this projection to give it length one. If Δ is the cone of monotone vectors, as in ordinal MDS, the cone projection is *monotone regression*. The fact that projection on the intersection of Δ and Σ is the same thing as normalizing the projection on Δ is due to De Leeuw (1975) and more generally (and more rigorously) to Bauschke, Bui, and Wang (2018). Since this step is the same as in standard MDS algorithms such as smacof (De Leeuw and Mair (2009), Mair, Groenen, and De Leeuw (2022)) we do not go into details here, and just refer to the literature.

2.2 Majorization

The second step (10b), minimizing over x for fixed current δ, is more complicated. There is no analytic solution, unlike the first step, and minimization requires an iterative process of its own. Thus, except in some very special cases, the second step requires an infinite number of "inner" iterations. Since implementing an infinite number of iterations is impossible we have to truncate the inner iteration sequence at some point. In our software we deviate from the strict ALS framework by not minimizing fStress over x for fixed δ, but by taking a single "inner" iteration and merely decrease fStress. If this is done judiciously we still obtain a convergent sequence of updates. This is similar to the strategy in other MDS algorithms such as smacof (De Leeuw (1977)) and alscal (Takane, Young, and De Leeuw (1977)).

In smacof the inner iteration step is a majorization step, by now more commonly known as an MM step. Briefly, we find a function $\kappa_f(x;y)$ such that

- $\kappa_f(x,y)\leq\sigma_f(x)$ for all x and y in Ξ, and
- $\kappa_f(x;y)=\sigma_f(x)$ if and only if $x=y$.

An inner iteration is of the form

$$x^{(\mu+1)}=\underset{x\in\Xi}{\operatorname{argmin}}\,\kappa(x,x^{(\mu)}).\tag{11}$$

If $x^{(\mu+1)}=x^{(\mu)}$ we declare convergence and stop. From (11),

$$\sigma_f(x^{(\mu+1)})\leq\kappa_f(x^{(\mu+1)},x^{(\mu)})<\kappa_f(x^{(\mu)},x^{(\mu)})=\sigma_f(x^{(\mu)}).\tag{12}$$

Thus an inner majorization step upgrading x for given δ decreases fStress (unless we stop). Remember that in ALS the inner iterative process (indexed by μ) goes on within step 2 of the "outer" update (indexed by ν).

In De Leeuw, Groenen, and Mair (2016) a majorization method was proposed to minimize rStress. It majorizes d^r as a function of x, and for this it uses the specific properties of the power function. It consequently cannot be used for fStress. The technique is incorporated in the smacofx R package, and numerical experience indicates it works, but convergence can be painfully slow. Because of the generality of the function f (any differentiable increasing function) it seems impossible to develop an majorization method for fStress along the same lines as the one for rStress. Consequently we go a somewhat different route in this paper. We do not only deviate from the strict ALS procedure by only partially minimizing loss over x for fixed d, we also deviate from the strict majorization approach by majorizing an approximation of fStress.

In deriving the approximation and majorization we surpress the dependency of fStress on δ, because in the second ALS substep δ is just a vector of constants. We first approximate $f(d(x))$ near $d(y)$ with

$$f(d_k(x))\approx f(d_k(y))+f;(d_k(y))(d_k(x)-d_k(y)).\tag{13}$$

Define

$$\omega_f(x;y):=\sum_{k=1}^K w_k(f(\delta_k)-f(d_k(y))-f'(d_k(y))(d_k(x)-d_k(y)))^2,\tag{14}$$

Note that in (13) the derivative is with respect to d, not with respect to x or y. This is a major difference with the majorizations in De Leeuw, Groenen, and Mair (2016).

Note that $\omega_f(x;x) = \sigma_f(x)$ for all x and if f is the identity then $\omega_f(x;y) = \sigma_f(x)$ for all x and y. If f is linear with intercept β and slope α then

$$\omega_f(x,y) = a^2 \sum_{k=1}^{K} w_k(\delta_k - d_k(x))^2 = a^2\sigma_r(x), \tag{15}$$

with $r = 1$.

We now give an alternative and more convenient expression for $\omega_f(x,y)$ from (14). Define

$$\tilde{w}_k(y) := w_k\{f'(d_k(y))\}^2, \tag{16a}$$

$$\tilde{\delta}_k(y) := \frac{f(\delta_k) - f(d_k(y))}{f'(d_k(y))} + d_k(y). \tag{16b}$$

Then

$$\omega_f(x;y) = \sum_{k=1}^{K} \tilde{w}_k(y)(\tilde{\delta}_k(y) - d_k(x))^2. \tag{17}$$

We see from (16a) that $\tilde{w}_k(y)$ is always non-negative. If f is convex then $\tilde{\delta}_k(y) \geq \delta_k \geq 0$. If f is concave then $\tilde{\delta}_k(y) \leq \delta_k$ and $\tilde{\delta}_k(y)$ can be negative. If $f(\delta_k) = f(d_k(y)) + f'(d_k(y))(\delta_k - d_k(y)) + o(|\delta_k - d_k(y)|)$ then $\tilde{\delta}_k(y) = \delta_k + o(|\delta_k - d_k(y)|)$. Also

$$\tilde{\delta}_k(y) = \left\{\frac{f'(\xi)}{f'(d_k(y))}\right\}\delta_k + \left\{1 - \frac{f'(\xi)}{f'(d_k(y))}\right\}d_k(y) \tag{18}$$

for some ξ in the interval between δ_k and $d_k(y)$.

For rStress equations (16a) and (16b) become

$$\tilde{w}_k(y) := r^2 w_k d_k^{2(r-1)}(y), \tag{19a}$$

$$\tilde{\delta}_k(y) := \frac{\delta_k^r + (r-1)d_k^r(y)}{r d_k^{r-1}(y)}. \tag{19b}$$

By convexity $\tilde{\delta}_k(y)$ is non-negative if $r \geq 1$. For $0 < r < 1$ we have $\tilde{\delta}_k(y) > 0$ if and only if $d_k(y) < (1-r)^r\delta_k$. If $\delta_k > d_k(y)$ then $\tilde{\delta}_k(y) > d_k(y) \geq 0$.

By writing $\omega_f(x,y)$ as in (17) we are on familiar majorization terrain. If $\delta_k(y) > 0$ we use Cauchy-Schwartz, as in standard smacof,

$$d_k(x) \geq \frac{1}{d_k(y)}\text{tr } x'A_k y, \tag{20a}$$

and if $\delta_k(y) < 0$ we use the arithmetic mean/geometric mean (AM/GM) inequality in the form

$$d_k(x) \leq \frac{1}{2}\frac{1}{d_k(y)}\{y'A_k y + x'A_k x\}, \tag{20b}$$

Use of the AM/GM inequality for majorizing terms with negative dissimilarities was pioneered by Heiser (1991).

Define the matrices

$$V(y) := \sum_{k=1}^{K} \tilde{w}_k(y) A_k, \tag{21a}$$

$$B(y) := \sum_{\tilde{\delta}_k(y)>0} \tilde{w}_k \frac{\tilde{\delta}_k(y)}{d_k(y)} A_k, \tag{21b}$$

$$H(y) := \sum_{\tilde{\delta}_k(y)<0} \tilde{w}_k \frac{|\tilde{\delta}_k(y)|}{d_k(y)} A_k. \tag{21c}$$

Note that all three matrices (more precisely matrix-valued functions) are positive semi-definite. Now, using these matrices and the inequalities (20a) and (20b),

$$\omega_f(x, y) \leq \kappa(x, y) := \gamma(y) + x'(V(y) + H(y))x - 2x'B(y)y, \tag{22}$$

with

$$\gamma(y) := \sum_{k=1}^{K} \tilde{w}_k(y)\tilde{\delta}_k^2(y) + y'H(y)y \tag{23}$$

which does not depend on x.

We now update using

$$x^{(\mu+1)} = \operatorname*{argmin}_x \kappa_f(x; x^{(\mu)}) = (V(x^{(\mu)}) + H(x^{(\mu)}))^+ B(x^{(\mu)})x^{(\mu)},$$

which results in

$$\sigma_f(x^{(\mu+1)}) \approx \omega_f(x^{(\mu+1)}; x^{(\mu)}) \leq \kappa_f(x^{(\mu+1)}; x^{(\mu)}) < \kappa_f(x^{(\mu)}; x^{(\mu)}) = \sigma_f(x^{(\mu)}).$$

Unlike in smacof there is no guarantee that σ_f decreases from one iteration to the other. Monotone convergence of loss function values will only happen if the approximation is good, i.e. if $d(x^{(\mu+1)})$ is close to $d_k(x^{(\mu)})$. This requires closer monitoring of convergence than in smacof. Also note that if $\tilde{\delta}_k(y) < 0$ for all k then $B(y) = 0$, which means that the update from y is equal to zero. Again, this eventuality must be monitored.

3 Software

The R function smacofSSRStressR() and the C function smacofSSRStressC() in the files smacofSSFStressR.R and smacofSSFStressC.c handle both metric and ordinal MDS minimizing fStress, in principle for an arbitrary f. The C code is compiled into a shared library smacofSSFStressC.so, which is loaded by the R front-end smacofSSFStressC.R

Both programs have input

```
function(theData,
         ndim = 2,
         xinit = NULL,
         ties = 1,
         itmax = 10000,
         usef = 1,
         eps = 1e-6,
         what = 0,
         rpow = 1,
         digits = 8,
         width = 10,
         verbose = TRUE,
         ordinal = FALSE)
```

Both functions output the same list

```
result <- list(
    delt = delt,
    dhat = dhat,
    dist = dnew,
    xmat = xnew,
    wmat = wght,
    loss = snew,
    ndim = ndim,
    init = xinit,
    itel = itel,
    nobj = nobj,
    iind = iind,
    jind = jind,
    ordi = ordinal,
    ties = ties,
```

```
    what = what,
    rpow = rpow,
    labl = labl,
    usef = usef,
    date = date()
  )
```

The input parameters are copied in the output so that the result has all the relevant information to reconstruct the run, and can pass it, for example, to the plot routines.

3.1 Data

For smacofSSRStressR.R and smacofSSRStressC.R the data and weights are in the MDS data format defined in De Leeuw (2025). This is a list with two scalars nobj and ndat, the number of objects and the number of data points (object pairs), and five vectors iind, jind, delta, blocks, weights of length ndat. Vectors iind and jind have integers between 1 and n with iind > jind that code for which pairs of objects we have dissimilarity information. The dissimilarities are in delta, the weights in weights. Dissimilarities are sorted in increasing order. The blocks vector codes the tie-blocks in the dissimilarities.

3.2 Initial Estimate

There are two possibilities. If xinit is not NULL then it is used as the initial estimate of the configuration. This is useful, for example, if we start the iterations for one f with the solution for another. If xinit is NULL then the function smacofSSRStressInit() computes an initial estimate using classical MDS (also known as Torgerson-Gower MDS) on the dissimilarities.

First we make sure the weights add up to one. If there are missing dissimilarities they are first imputed by setting them equal to the weighted mean of the non-missing dissimilarities. Then we double center the squared δ and compute the classical scaling solution using the eigs_sym() function from the RSpectra package (Qiu and Mei (2024)).

It is well-known that if the fit is half-way decent then the distances in the Torgerson-Gower solution underestimate the dissimilarities. It makes sense, consequently, to multiply the initial configuration by a constant to get a better fit (Malone, Tarazaga, and Trosset (2002)). We use the fStress approximation (7) and minimize

$$\sigma(\lambda) = \sum_{k=1}^{K} w_k \{f'(\delta)\}^2 (\delta_k - d_k(\lambda x))^2$$

over λ.

3.3 Functions

The R and C versions come with the following hard-coded functions. The *what* (zero to four) selects the function, the *rpow* parameter gives the r parameter value.

Table 1: f examples

	f(x)	f'(x)	f^{-1}(y)
power	x^r	rx^{r - 1}	y^{1 / r}
shifted log	log(rx + 1)	r/(rx+1)	(exp(y) - 1) / r
shifted exponent	exp(rx) - 1	r exp(rx)	log(y + 1)/r
log	rlog(x)	r/x	exp(y/r)
atan	atan(rx)	r / (1 + (rx)^2)	tan(y)/r

The selection code is in the C function smacofSSFStressFlist() and in the R function smcofSSFStressSelect(). It is easy to add functions to both versions, but adding them to the C version requires recompiling of the shared library.

3.4 Main

The arguments of smacofSSFStressR() and smacofSSFStressR() are

Thus fStress() needs an initial configuration. Parameters f and g are functions that compute the transformation f and its derivative. The file fStress.R includes flog, glog, fexp, gexp, and fPower, gPower. Parameter r is included in case f and g need additional arguments (as in the case of fPower and gPower). Since all code is in R, it is easy to add additional transformation functions if needed. The output of fStress() is

```
list(
    delt = delta,
    dhat = delf,
    wmat = wght,
    conf = xnew,
    dist = dnew,
    loss = snew,
    itel = itel
  )
```

The arguments of smacofSSRStress() are

The C code follows the conventions of the smacof versions in De Leeuw (2025). There is only a single parameter $rpow$ for the transformation. The output of smacofSSRStress() is

```
list(
    delt = theData$delta,
    dhat = h$dhat,
    dist = h$edis,
    xmat = matrix(h$xnew, nobj, ndim),
    wmat = h$wght,
    loss = h$snew,
    ndim = ndim,
    init = xinit,
    itel = h$itel,
    nobj = nobj,
    iind = h$iind,
    jind = h$jind,
    weighted = weighted,
    ordi = ordinal,
    ties = h$ties
  )
```

3.5 Plots

The four standard plots that come with the recent smacof programs () are

- smacofConfigurationPlot()
- smacofShepardPlot()
- smacofDistDhatPlot()
- smacofResidualPlot()

The output list of smacofSSFStress() can be fed immediately into these four plot routines. They only have side effects (i.e. make a plot) and return NULL.

3.5.1 smacofConfigurationPlot()

It is easy to explain what smacofConfigurationPlot() does. It plots a two-dimensional projection of the configuration. Its arguments are self-explanatory.

```
function(h, labels = NULL, dim1 = 1,
         dim2 = 2, pch = 16, col = "RED", cex = 1)
```

3.5.2 smacofShepardPlot()

A Shepard plot (De Leeuw and Mair (2015)) traditionally plots observed dissimilarities on the horizontal axis, and both disparities and distances on the vertical axis. The (δ_k^0, δ_k) points are connected by a straight line through the origin in the metric case and by an increasing step function in the ordinal case. The $(delta, dist)$ points are optionally connected with vertical "fitlines" to the corresponding $(delta, dhat)$ points.

```
function(h, main = "ShepardPlot", fitlines = TRUE, colline = "RED",
    colpoint = "BLUE", type = 0, lwd = 2, cex = 1, pch = 16)
```

In the fStress solution it makes sense to put the final δ and $f(d(x))$ on the vertical axis, but there is some ambiguity on what we use on the horizontal axis. As explained in the introduction in metric MDS we can have $\delta \propto f(\delta^0)$, with δ^0 observed dissimilarities. In that case we put $f(\delta^0)$ on the horizontal axis, and the the points $f(\delta_k^0), \delta_k$ are on a straight line.

In the numerical case do we analyze $\delta = f(\delta^0)$ or $\delta = \delta^0$. In the ordinal case: it does not make a difference, except for the plots

In the fStress context there is an extra arguments "type". Remember that the list h has both a "what" and an "rpow" element. The "type" argument is either one or zero. If it is zero (the default) then we plot $f(\delta_k)$ on the horizontal axis and $\hat{d}_k$ and $f(d_k(x))$ on the vertical axis. If "type" is equal to one then we plot δ_k on the horizontal axis and $f^{-1}(\hat{d}_k)$ and $d_k(x)$ on the vertical axis.

3.5.3 smacofDistDhatPlot()

smacofDistDhatPlot() has arguments

```
function(h, fitlines = TRUE, colline = "RED", colpoint = "BLUE",
    main = "Dist-Dhat Plot", type = 0, cex = 1, lwd = 2, pch = 16)
```

The "type" parameter works in the same way as in the Shepard plot.

3.5.4 smacofResidualPlot()

smacofResidualPlot() is more complicated. It has arguments

```
function (h, main = "ResidualPlot", probs = seq(0, 1, 0.25),
    whatres = 1, whatplot = 1, type = 0, qlines = TRUE, colpoints = "RED",
    collines = "BLUE", lwdpoints = 2, lwdlines = 2, cex = 1,
    pch = 16)
```

As in the previous plots *type* computes residuals as

whatplot can have the values *ecdf* (default) or *density*. Computation is done with the ecdf() and density() functions from the stats package.

whatres chooses to plot the residual (default), its absolute value, or its square

4 Examples

We analyse the colour data from Ekman (1954) with different functions f, using both smacofSSFStressR() and using smacofSSFStressC().

Table 2: smacofSSFStressR() results Ekman Data

	itel	fStress
logarithm	154	0.0041809
power .1	804	0.0002572
power .5	227	0.0046666
power 1	35	0.0116012
power 2	49	0.0182351
power 5	793	0.0080275
exponent	36	0.0451131

Table 3: smacofSSFStressC() results Ekman Data, metric MDS

	itel	fStress
logarithm	206	0.0057918
power .1	854	0.0002733
power .5	237	0.0059543
power 1	35	0.0172132
power 2	50	0.0328807
power 5	807	0.0203153
exponent	32	0.0207335

We see that for the power transforms the R and C functions arrive at the same solution. The number of iterations is not exactly the same, because of differences in the initial configuration. With smacofSSRStress() we can also do ordinal MDS.

Table 4: smacofSSFStressC() results Ekman Data, ordinal MDS

	itel	fStress
logarithm	336	0.0002466
power .1	975	0.0000066
power .5	207	0.0001651
power 1	141	0.0005337

	itel	fStress
power 2	233	0.0009015
power 5	2638	0.0011002
exponential	160	0.0007895

The package microbenchmark (Mersmann (2024)) can be used to compare the R and C versions. For $r = 1$ the C version is about ten times faster, for $r = .1$ and $r = 5$ it is more like 30 times faster.

Table 5: R and C timings Ekman Data in microseconds, metric MDS

	R times	C times
logarithm	24030	690
power .1	101871	3115
power .5	27919	1073
power 1	3967	382
power 2	5929	474
power 5	95857	2920
exponential	3883	349

Table 6: R and C timings Ekman Data in milliseconds, nonmetric MDS

	R times	C times
logarithm	111.93	2.83
power .1	420.03	8.96
power .5	115.90	2.15
power 1	58.00	1.46
power 2	95.26	2.38
power 5	1172.62	23.53
exponential	63.57	1.49

References

Bauschke, Heinz H., Minh N. Bui, and Xianfu Wang. 2018. "Projecting onto the Intersection of a Cone and a Sphere." *SIAM Journal on Optimization* 28: 2158–88.

De Leeuw, Jan. 1975. "A Normalized Cone Regression Approach to Alternating Least Squares Algorithms." Department of Data Theory FSW/RUL. https://jansweb.netlify.app/publication/deleeuw-u-75-a/deleeuw-u-75-a.pdf.

———. 1977. "Applications of Convex Analysis to Multidimensional Scaling." In *Recent Developments in Statistics*, edited by J. R. Barra, F. Brodeau, G. Romier, and B. Van Cutsem, 133–45. Amsterdam, The Netherlands: North Holland Publishing Company.

———. 1984. "Differentiability of Kruskal's Stress at a Local Minimum." *Psychometrika* 49: 111–13.

———. 2017. "Higher Partials of fStress. Who Needs Them ?" 2017. https://jansweb.netlify.app/publication/deleeuw-e-17-r/deleeuw-e-17-r.pdf.

———. 2025. "Yet Another Smacof - Square Symmetric Case." 2025. https://jansweb.netlify.app/publication/deleeuw-e-25-b/.

De Leeuw, Jan, Patrick Groenen, and Patrick Mair. 2016. "Minimizing rStress Using Majorization." 2016. https://jansweb.netlify.app/publication/deleeuw-groenen-mair-e-16-a/deleeuw-groenen-mair-e-16-a.pdf.

De Leeuw, Jan, and Patrick Mair. 2009. "Multidimensional Scaling Using Majorization: SMACOF in R." *Journal of Statistical Software* 31 (3): 1–30. https://www.jstatsoft.org/article/view/v031i03.

———. 2015. "Shepard Diagram." In *Wiley StatsRef: Statistics Reference Online*, 1–3. Wiley.

Ekman, Gosta. 1954. "Dimensions of Color Vision." *Journal of Psychology* 38: 467–74.

Groenen, Patrick J. F., Jan De Leeuw, and Rudolf Mathar. 1995. "Least Squares Multidimensional Scaling with Transformed Distances." In *From Data to Knowledge: Theoretical and Practical Aspects of Classification, Data Analysis and Knowledge Organization*, edited by W. Gaul and D. Pfeifer. Berlin, Germany: Springer Verlag. https://jansweb.netlify.app/publication/groenen-deleeuw-mathar-c-95/groenen-deleeuw-mathar-c-95.pdf.

Groenen, Patrick J. F., and Michel Van de Velden. 2016. "Multidimensional Scaling by Majorization: A Review." *Journal of Statistical Software* 73 (8): 1–26. https://www.jstatsoft.org/index.php/jss/article/view/v073i08.

Heiser, Willem J. 1991. "A Generalized Majorization Method for Least Squares Multidimensional Scaling of Pseudodistances that May Be Negative." *Psychometrika* 56 (1): 7–27.

Kruskal, Joseph B. 1964a. "Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis." *Psychometrika* 29: 1–27.

———. 1964b. "Nonmetric Multidimensional Scaling: a Numerical Method." *Psychometrika* 29: 115–29.

Mair, Patrick, Patrick J. F. Groenen, and Jan De Leeuw. 2022. "More on Multidimensional

Scaling in R: smacof Version 2." *Journal of Statistical Software* 102 (10): 1–47. https://www.jstatsoft.org/article/view/v102i10.

Malone, Samuel W., Pablo Tarazaga, and Michael W. Trosset. 2002. "Better Initial Configurations for Metric Multidimensional Scaling." *Computational Statistics and Data Analysis* 41: 143–56.

Mersmann, Olaf. 2024. *microbenchmark: Accurate Timing Functions*. https://CRAN.R-project.org/package=microbenchmark.

Qiu, Yixuan, and Jiali Mei. 2024. *RSpectra: Solvers for Large-Scale Eigenvalue and SVD Problems*. https://CRAN.R-project.org/package=RSpectra.

Ramsay, James O. 1977. "Maximum Likelihood Estimation in Multidimensional Scaling." *Psychometrika* 42: 241–66.

Rusch, Thomas, Jan De Leeuw, Patrick Mair, and Kurt Hornik. In Press. "Flexible Multidimensional Scaling with the r Package Smacofx." *Journal of Statistical Software*, In Press. https://jansweb.netlify.app/publication/rusch-deleeuw-mair-hornik-a-25/rusch-deleeuw-mair-hornik-a-25.pdf.

Takane, Yoshio, Forrest W. Young, and Jan De Leeuw. 1977. "Nonmetric Individual Differences in Multidimensional Scaling: An Alternating Least Squares Method with Optimal Scaling Features." *Psychometrika* 42: 7–67.